AI-Native Grocery Merchant

Document 03 — System Architecture & Technical Design

Status	LOCKED — Development Source of Truth
Depends On	Document 01 — Development Master Specification; Document 02 — Functional Requirements
Purpose	Define technical architecture, component boundaries, data flow and trust boundaries
Scope	Hackathon MVP; implementation-focused, not production-scale

Architecture invariant: AI is intelligent but untrusted. The backend is authoritative. The policy engine is the financial authorization boundary. Razorpay is the payment execution rail.

1. Architecture Goals

- Enable an AI buyer to perform meaningful grocery procurement work rather than simple product lookup.
- Keep all money, inventory and authorization decisions server-authoritative.
- Separate probabilistic AI reasoning from deterministic transaction authorization.
- Support two optimization strategies: Best Value and Best Quality Within Budget.
- Support smart voucher and loyalty decisions without turning the system into a coupon marketplace.
- Create a reliable Razorpay Test Mode payment path.
- Make the entire financial path explainable and auditable.
- Keep the implementation small enough to build, test and demonstrate reliably.

2. High-Level Architecture

```
USER
■
▼
FRONTEND
■
▼
BACKEND API / ORCHESTRATOR
■
■■■ AI AGENT
■ ■■■ Intent / Requirement Extraction
■ ■■■ Catalog Research
■ ■■■ Quality / Evidence Analysis
■ ■■■ Quantity + Pack Optimization
■ ■■■ Deal Optimization
■ ■■■ Voucher + Loyalty Reasoning
■
■■■ COMMERCE CORE
■ ■■■ Catalog
■ ■■■ Cart
■ ■■■ Order State
■
■■■ INCENTIVE SERVICES
■ ■■■ Voucher Eligibility
■ ■■■ Loyalty Eligibility
■
■■■ POLICY ENGINE
■ ■■■ Deterministic ALLOW / DENY
■
■■■ PAYMENT ADAPTER
■ ■■■ Razorpay Test Mode
■
■■■ AUDIT SERVICE

RAZORPAY → WEBHOOK → BACKEND → ORDER / PAYMENT STATE → AUDIT
```

The AI may request data and propose a transaction. It never directly controls the payment rail.

3. Component Responsibilities

Component	Primary responsibility	Authority
Frontend	User interaction, goal entry, mandate display, basket comparison, selection and payment UI	Not authoritative for financial state
Backend API	Authentication/session context, orchestration, validation and server-side state access	Authoritative application boundary
AI Agent	Reasoning, research synthesis, optimization, recommendation and recovery	Untrusted; proposal only
Commerce Core	Catalog, product facts, cart, authoritative totals and order state	Authoritative for commerce state
Optimization Layer	Quantity, pack, cost and basket calculations; deterministic calculations where practical	Authoritative for calculations it owns
Incentive Layer	Voucher/loyalty eligibility, discount/reward calculations and state	Authoritative for incentive validity
Policy Engine	Mandate validation and transaction authorization	Final authority before money action
Payment Adapter	Razorpay order creation, verification and webhook mapping	Authority for external payment state mapping
Audit Service	Financial/authorization event records	Record-only; does not authorize

4. Trust Boundaries

The architecture intentionally places a hard trust boundary between AI reasoning and financial execution.

```
UNTRUSTED AI
■
■ structured proposal / tool request
▼
TRUSTED BACKEND
■
■ authoritative data + deterministic validation
▼
POLICY ENGINE
■
■■■ DENY → no money action
■■■ ALLOW → payment adapter
↓
RAZORPAY
```

- The AI cannot modify the mandate.
- The AI cannot set the authoritative price.
- The AI cannot declare stock available.
- The AI cannot determine the authoritative final payable amount.
- The AI cannot bypass policy denial.
- The browser cannot directly create an authorized Razorpay order.
- The browser cannot be trusted to report payment success.
- The final checkout state is revalidated immediately before payment.

5. End-to-End Data Flow

5.1 Planning flow

```
USER GOAL → INTENT → REQUIREMENTS → CATALOG CANDIDATES → QUALITY EVIDENCE → QUANTITY/PACK OPTIONS → DEAL OPTIONS → INCENTIVE OPTIONS → BEST VALUE + BEST QUALITY → AI RECOMMENDATION → USER SELECTION
```

5.2 Authorization and payment flow

```
SELECTED BASKET → SERVER RECONSTRUCTION → PRICE/STOCK/INCENTIVE REVALIDATION → POLICY ENGINE → ALLOW/DENY → IF ALLOW: CREATE RAZORPAY ORDER → CHECKOUT → PAYMENT → SERVER VERIFICATION/WEBHOOK → ORDER CONFIRMED → AUDIT
```

5.3 Failure recovery flow

```
AI PROPOSAL → POLICY DENY → STRUCTURED REASON → AI RE-OPTIMIZATION → NEW BASKET → POLICY RECHECK → ALLOW OR SAFE STOP
```

6. AI Agent Architecture

The AI agent is a planning and optimization layer over controlled backend tools. It should not receive unrestricted database or payment access.

Tool class	Purpose	Returns
Intent parser	Normalize user goal, budget and constraints	Structured intent
Catalog search	Find matching products	Authoritative product candidates
Product evidence	Retrieve controlled quality/review evidence	Evidence records
Pack optimizer	Evaluate pack combinations	Quantity/cost candidates
Incentive lookup	Retrieve eligible voucher/reward candidates	Incentive candidates
Basket optimizer	Construct Best Value / Best Quality	Structured basket proposals
Policy check	Ask backend to validate a proposal	ALLOW/DENY + rule/reason
Checkout request	Request final payment after authorization	Server-managed payment initiation
Audit read	Retrieve transaction history where needed	Audit events

7. Commerce Core

The Commerce Core owns all authoritative merchant and transaction data.

- Catalog products have stable identifiers.
- Price and stock are backend-owned.
- Pack size and unit definitions are backend-owned.
- Cart state is server-owned.
- Gross total, discount total and final payable are calculated server-side.
- Order state is independent of AI conversation state.
- AI explanations may reference commerce facts but cannot mutate authoritative facts through text.

8. Optimization Architecture

Optimization should be split so that arithmetic and constraint checks are deterministic wherever practical, while the LLM handles semantic interpretation and evidence synthesis.

Concern	Preferred implementation boundary
Requirement semantics	AI
Candidate retrieval	Backend/catalog service
Quantity arithmetic	Deterministic service
Pack combination search	Deterministic algorithm/service, optionally orchestrated by AI
Discount arithmetic	Deterministic incentive service
Voucher eligibility	Backend/incentive service
Loyalty eligibility	Backend/incentive service
Quality evidence synthesis	AI over controlled evidence
Basket objective comparison	Deterministic scoring plus AI explanation
Final payable	Backend only

9. Best Value vs Best Quality

Best Value: minimize practical cost subject to required quantity, acceptable quality and all constraints.
Best Quality: maximize practical quality/reliability subject to required quantity and hard budget.

The two baskets should be generated from the same authoritative candidate universe. Their difference must arise from objective weighting, not from fabricated products or prices.

The AI then recommends one basket based on the stated context, but the user selects the final option. User selection is a preference decision, not authorization to bypass policy.

10. Incentive Architecture

Vouchers and loyalty rewards are modeled as merchant-controlled incentives. The AI can reason about whether to consume or preserve them, while the backend owns eligibility and arithmetic.

Layer	Responsibility
AI	Evaluate current benefit vs preservation/future value; explain USE/SAVE recommendation.
Incentive service	Validate eligibility, expiry, thresholds and discount/reward calculation.
Policy engine	Ensure selected incentive is valid and allowed in the final transaction.
Audit	Record incentive candidate, decision, value and final application state.

Example: ■100 basket + 5% voucher = ■5 saving. If the voucher has meaningful future utility, the AI may recommend SAVE. The system must never add unnecessary products merely to unlock the voucher.

11. Mandate and Policy Architecture

The mandate is server-owned authorization context. The policy engine receives the selected final basket and authoritative state and returns a deterministic result.

```
MANDATE_VALID → CATEGORY_ALLOWED → STOCK_AVAILABLE → MAX_PER_ITEM → INCENTIVE_VALIDITY/ELIGIBILITY →
FINAL_AMOUNT_CALCULATION → MAX_SPEND → ALLOW/DENY
```

The policy engine must not depend on LLM confidence, natural-language claims or browser-supplied totals.

12. Cart and Order State

State	Meaning
OPEN_CART	Basket is being assembled; no payment created.
POLICY_PENDING	Final basket awaiting policy authorization.
AUTHORIZED	Policy has allowed the current transaction state.
PAYMENT_PENDING	Razorpay order exists; payment not yet authoritative as successful.
PAID	Server has verified successful payment state.
PAYMENT_FAILED	Payment attempt failed or was not completed.
CANCELLED	Transaction was cancelled before completion.

13. Razorpay Integration Boundary

The payment adapter is the only application component that should initiate the Razorpay order. It receives an already-authorized, server-calculated amount.

- Never pass an LLM-generated amount directly to Razorpay.
- Never trust a browser-calculated amount as authoritative.
- Create the Razorpay order only after final policy ALLOW.
- Keep Razorpay identifiers mapped to internal order/transaction identifiers.
- Verify payment state on the server.
- Process webhooks idempotently.
- Keep Test Mode credentials in environment configuration, never source code.

14. Audit Architecture

Audit records should be append-oriented. An audit event should describe what happened, who/what initiated it, what decision was made and the relevant financial context.

```
INTENT_RECEIVED
REQUIREMENTS_CREATED
PRODUCT_RESEARCHED
BASKET_CREATED
VOUCHER_EVALUATED
LOYALTY_EVALUATED
BASKET_RECOMMENDED
BASKET_SELECTED
POLICY_ALLOW / POLICY_DENY
RAZORPAY_ORDER_CREATED
```

```
PAYMENT_VERIFIED / PAYMENT_FAILED
ORDER_CONFIRMED
```

15. API Boundary Model

The API should expose business capabilities rather than raw database operations wherever practical.

Endpoint family	Primary responsibility
GET /catalog	Browse authoritative catalog.
GET /catalog/search	Search candidate products.
GET /mandate/:agentId	Read current mandate.
POST /agent/plan	Create/retrieve structured AI planning result.
POST /basket/select	Record user's selected basket and trigger revalidation.
GET /cart/:agentId	Read server cart.
POST /cart/:agentId/add	Add authorized item after validation.
POST /cart/:agentId/remove	Remove item and recalculate.
POST /checkout/:agentId	Final validation + policy + Razorpay order creation.
POST /webhook/razorpay	Receive and reconcile Razorpay events.
GET /audit/:agentId	Read transaction audit history.

16. Security Requirements

- Validate every request at the backend boundary.
- Do not expose secret keys or server credentials to the frontend.
- Treat all LLM output as untrusted input.
- Use allowlisted tools and structured schemas for agent actions.
- Enforce mandate and policy checks server-side.
- Use stable internal identifiers instead of accepting arbitrary product prices from clients.
- Prevent duplicate payment processing through idempotency/state checks.
- Validate webhook authenticity according to the payment integration design.
- Do not log secrets, authentication tokens or sensitive credentials.
- Keep audit data tamper-resistant within the practical scope of the MVP.

17. Failure and Recovery Architecture

Failures should be represented as structured domain errors so the AI can reason about them without being given authority to override them.

Failure	Backend result	AI behavior
Budget exceeded	DENY / MAX_SPEND	Re-optimize within existing mandate.
Category prohibited	DENY / CATEGORY_ALLOWED	Remove or replace item.
Out of stock	INVALID / STOCK_AVAILABLE	Find alternative.
Voucher invalid	INVALID / INCENTIVE	Recalculate without voucher.
Price changed	REVALIDATE_REQUIRED	Rebuild/recalculate basket.
Payment failed	PAYMENT_FAILED	Explain state; allow safe retry where supported.

18. Data Consistency Rules

- A product ID is the identity; price is fetched from authoritative current state.
- Cart totals are derived, not trusted from client state.
- Voucher and loyalty application must be reproducible from backend rules.

- A basket shown to the user may become stale; checkout must revalidate it.
- Payment state must be derived from authoritative verification/webhook information.
- Audit entries should reference stable transaction, order, mandate and agent identifiers.
- AI conversation history is not the source of truth for transaction state.

19. Deployment / Environment Boundary

The MVP should support at least a development environment and a test/demo environment using environment variables for configuration.

Configuration	Handling
AI provider/API key	Environment secret.
Razorpay Test Key ID/Secret	Environment secret; Test Mode only.
Database URL	Environment configuration/secret.
Webhook secret	Environment secret.
Application URL	Environment configuration.
Feature flags	Environment/configuration where needed; no hidden production-only behavior.

20. Observability Requirements

- Every transaction should have a correlation/transaction ID.
- AI proposal, policy result, order ID and payment ID should be traceable.
- Errors should include structured error codes.
- Logs should distinguish AI planning failures from policy denials and payment failures.
- Do not use verbose raw LLM logs as a substitute for business audit events.

21. Architectural Non-Negotiables

- LLM cannot authorize itself.
- LLM cannot change the mandate.
- Frontend cannot authorize itself.
- Frontend cannot set the authoritative payment amount.
- Policy must run before payment order creation.
- Final basket must be server-revalidated before payment.
- Razorpay order creation is backend-only.
- Payment success requires authoritative verification.
- Voucher/reward eligibility is backend-authoritative.
- Stock and price are backend-authoritative.
- Policy denial cannot be overridden by prompt wording or client parameters.
- The system must be able to demonstrate a graceful policy-denial recovery.

22. Recommended MVP Build Shape

Use the simplest architecture that preserves the trust boundaries. A modular monolith is preferred over unnecessary microservices for this MVP.

Module	Suggested boundary
api	HTTP routes, validation, request context.
agent	AI orchestration, tools, structured planning output.
catalog	Products, search, stock and product evidence.
optimization	Quantity, pack, basket scoring and comparisons.

Module	Suggested boundary
incentives	Voucher and loyalty rules/calculation.
policy	Mandate + deterministic authorization.
cart	Cart state and totals.
payments	Razorpay adapter, verification and webhook handling.
audit	Audit event creation/query.
db	Persistence layer and migrations.

23. Architecture Definition of Done

- Every component has a clear responsibility and does not duplicate authority owned elsewhere.
- AI communicates through controlled backend tools rather than direct payment/database access.
- The selected basket can be reconstructed and recalculated server-side.
- Policy can deterministically ALLOW or DENY the final transaction.
- No payment order is created before policy ALLOW.
- Razorpay Test Mode payment state can be reconciled.
- A complete transaction can be traced through audit records.
- The budget-denial recovery path works without changing the mandate.
- The architecture can support both Best Value and Best Quality without duplicating commerce state.

24. Document Authority

Document 03 defines technical boundaries and data flow. Document 01 remains the highest-level locked MVP scope. Document 02 defines functional behavior. Later documents will define AI internals, optimization formulas, policy details, data schemas, API contracts, payment implementation and tests in greater detail.

Locked architecture summary: AI reasons → backend validates → policy authorizes → Razorpay executes → audit records.